2026 年第八届

全球校园人工智能算法精英大赛

算法创新赛

（请填写赛题名称）

技术报告

团队名称：（请填写）

参赛编号：AIC-2026-（请填写编号）

作品名称：研海寻踪：多智能体科研知识图谱平台

日期：2026 年 10 月

目　录

作品简介 3
一、项目概述 3
（一）项目背景与意义 3
1. 论文在涨，“核对”的成本没有跟着降 3
2. 通用大模型的结论，回不到原文 3
（二）赛题方向定位 3
（三）现有工具的位置与缺口 3
二、总体设计 4
（一）设计原则 4
（二）系统结构 4
（三）数据流与两条底线 5
三、核心技术 6
（一）证据对象与字符级跨度 6
（二）三道分工与确定性裁判 6
（三）候选关系的四条可验证接收条件 6
（四）全文知识库：只读视图与规模 7
（五）产品化：八页工作区与数据模型 7
四、实现与验证 8
（一）实现规模 8
（二）验证方式 9
（三）当前的测量进度与已知问题 9
五、创新点 10
（一）把“接收一条关系”拆成四条可验证的判据 10
（二）判定环节的确定性边界 10
（三）两种判断在结构上分开 10
（四）把负结果留在方案里 10
六、应用前景与商业化路径 10
（一）目标用户 10
（二）成本结构 11
（三）推广路径 11
七、附录 11
（一）运行方式 11
（二）目录结构与主要模块 11
（三）数据与口径说明 11

作品简介

研海寻踪把一篇论文变成一张能回到原文的关系图。系统从全文里按字符切出证据片段，再由提出者、批判者、裁判三道分工判断“这段文字能否支撑这条关系”；裁判只做确定性计算，同样的输入必得同样的分数，任何人都能重跑。每条关系都挂着证据编号与偏移，点开就能看到原文那一行。团队自建了五个领域的开放全文知识库，已抽出 58,801 条证据与 30,918 条候选关系；八页工作区覆盖导入、审阅、建图、核证到汇总结论的全过程，用户的判断与机器的判断分开存放，导出带版本号。产品已交付 Windows 单文件桌面版与手机版，断网时也能用本机知识库。

（本段 265 字，要求不超过 300 字。）

一、项目概述

（一）项目背景与意义

1. 论文在涨，“核对”的成本没有跟着降

研究者读一篇论文，真正花时间的地方不是读完，而是回头核对：这句话是谁说的、依据在哪一段、结论和原文是不是同一个意思。检索工具能把论文找出来，笔记工具能把句子存下来，但“这条结论出自原文哪一行”这件事，目前基本靠人工重新翻。

当语料从几十篇涨到几百篇，这件事的代价会非线性上升：越依赖二手摘要，越难回到一手证据；越难回到证据，结论的可信度就越只能靠作者自觉。

2. 通用大模型的结论，回不到原文

把综述任务交给通用大模型，能很快得到一段读起来通顺的结论，但这段结论通常不可核验：它没有给出字符级出处，同一问题问两次答案可能不同，也无法说明哪一条依据被推翻过。对科研场景，这三点中的任何一点都足以让结论只能当草稿。

本项目要解决的就是这一步：让每一条关系都能被逐字切回原文，让判断过程可以被重跑，让用户自己的取舍与机器的判断分开存放。

（二）赛题方向定位

本项目属于算法创新赛的“科研知识图谱与智能体协同”方向：以算法（证据切分、实体与关系抽取、候选关系的可验证接收条件、确定性裁决）为主体，以智能体分工为组织形式，以可运行的产品为交付形态。算法部分的每一处取舍都能在仓库里找到实现与验证，智能体部分不做“多问几个模型取多数”式的堆叠。

（三）现有工具的位置与缺口

把常用工具放在同一张表里看，缺口是清楚的：

表 1 现有工具与本项目的位置

工具类型	能做什么	缺什么
文献管理器（如 Zotero）	收藏、题录、批注、PDF 阅读	不会从正文里抽出关系，也不做证据判定
文献问答产品	给一段摘要式回答	答案回不到具体字符位置，判断不可重跑
通用大模型	生成综述式结论	无字符级出处；同一输入两次结果可能不同
图谱构建工具	按人工给定的模板抽实体关系	判定环节依赖模型，缺少可复现的裁决口径
研海寻踪	抽取 + 分工裁决 + 证据跨度 + 八页工作区	语料规模与人工标注仍在推进（见第四节）

二、总体设计

（一）设计原则

四条原则决定了后面所有技术选择，也决定了哪些功能我们刻意不做：

· 证据优先：没有带跨度的证据，就不产出关系；取不到原文时不拼凑、不让模型补写。

· 判断可重跑：裁决环节是确定性代码，同样的输入必得同样的分数与状态。

· 两种判断分开放：机器给的是“机器状态”，用户给的是“我的决定”，分表存放、导出时各自成列。

· 只展示做过的判断：图谱只画用户已经纳入的关系，未判的留在队列里，不用机器状态冒充结论。

（二）系统结构

系统分五层，自下而上是数据层、抽取层、裁决层、图谱层、产品层。上层只能读下层已经落好的东西，不能越级去调用模型或直接改库，这也是“证据优先”能被结构保证、而不只是写在文档里的原因。



图 1 五层结构：语料先进证据层，裁决只认带跨度的证据，产品只展示可回原文的关系

（三）数据流与两条底线

从一份开放全文到图谱里的一条边，中间要经过六步：登记语料（许可与指纹）→ 切段落（页码与偏移）→ 切证据（字符级跨度）→ 生成候选关系（实体与触发词）→ 三道分工裁决 → 写入图谱与结论。每一步的产物都有编号，任何一条关系都能顺着编号倒推回原文。



图 2 证据链数据流，以及两条不妥协的底线

三、核心技术

（一）证据对象与字符级跨度

证据不是一个句子字符串，而是一个带坐标的对象：论文编号、段落编号、页码、字符起止、以及切分时的文本指纹。判定关系时只认这个对象；渲染时按坐标从原文里切片，所以页面上看到的永远是原文那一行，而不是抽取阶段留下来的副本。

这条设计带来一个直接后果：当原文没有保存（用户未授权保存正文）时，上下文查询会明确返回“取不到”，而不是悄悄退回一段近似文本。我们宁可让页面缺一块，也不让读者以为看到了原文。

（二）三道分工与确定性裁判

提出者负责从证据出发提候选关系，必须指明证据编号；批判者按九条结构化检查逐条挑毛病（证据是否在场、方向是否写反、关系类型是否成立、跨度是否跨句等）；裁判只做确定性计算——查知识库、比类型约束、按规则算分，不调用任何模型。



图 3 三道分工的职责边界，以及我们量到的负结果

（三）候选关系的四条可验证接收条件

为了让“接收”这件事可复核，我们把接收条件拆成四条可以分别验证的判据：证据有效、实体定位唯一、关系类型与方向合规、两个实体之间存在谓词。四条全过才是“接收”，证据无效直接“驳回”，其余进“待人判定”。

严格判据会把接收量压得很低，所以我们把它做成可选口径而不是默认值，每条关系的两种状态都留在库里，谁想用严格口径都能自己算一遍。

（四）全文知识库：只读视图与规模

课题里的证据默认只来自用户自己导入的论文；此外团队自建了一份开放全文知识库，覆盖五个领域，以 SQLite 只读方式接入（`mode=ro`），用来做离线复现、批量评测，也可以整领域接进某个课题。界面不写库、不改库——抽取结果被重建时，不会因为读写冲突出现半夜的空图。当前库的实测规模如下。

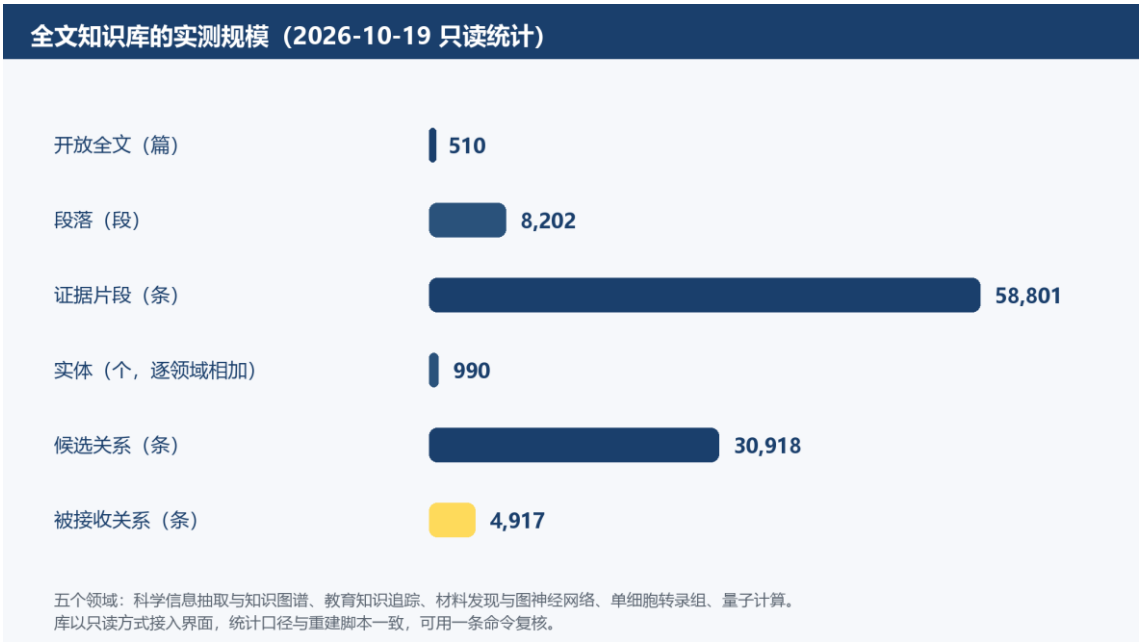


图 4 全文知识库的实测规模（2026-10-19 只读统计）

表 2 五个领域的语料与库内构成

领域	题录	本地开放全文	证据片段	候选关系
科学信息抽取与知识图谱	172	150	12,734	3,370
教育知识追踪与个性化学习	130	100	8,400	3,008
材料发现与图神经网络	129	100	13,207	9,224
量子计算与量子信息处理	180	80	12,354	5,075
单细胞转录组数据分析	180	80	12,106	10,241
合计	791	510	58,801	30,918

（五）产品化：八页工作区与数据模型

界面不是把接口排一遍，而是按研究者真实动作排成一条动线：先看还差什么（课题概览），再跑一次抽取（研究工作台），把论文进来（文献库），逐条审关系（构建图谱），只看留下的关系（知识图谱），需要时回原文核对（证据中心），把结论与线索合起来看（研究发现），最后共享、导出、留版本（设置）。



图 5 八页工作区与每一页负责的动作

数据模型上，课题的论文归属、候选关系、关系证据、用户决定、图谱版本、线索、成员权限各自成表，其中“机器状态”与“用户决定”分属不同表，任何一方都不覆盖另一方。当前库结构版本为 7，迁移只做增量，不回填历史数据。

四、实现与验证

（一）实现规模

后端是单一 FastAPI 服务（默认端口 8766），前端为 React + TypeScript 构建后由后端直接托管；另有一个面向内部实验的内部服务单独起在 8767，用户端不暴露它的任何接口。仓库内测试 600 余项，覆盖率门槛 75% 已接进提交前检查；整套检查包含编译、测试、覆盖率、前端类型检查与构建。



图 6 工程状态：测试、知识库、工作区与发行形态

（二）验证方式

我们对“做完”的定义是可复现，所以每一类结论都配一条命令：

· 接口与业务逻辑：600 余项后端测试，覆盖登录、课题、共享权限、知识库接入与内部服务权限。

· 界面：无头浏览器在 1440 / 1024 / 390 / 320 四档宽度逐页截图，逐页判横向溢出，脚本记录 scrollWidth 与 innerWidth 是否相等。

· 发行包：单文件程序带 `--check` 自检，起服务后依次打健康检查、登录、知识库、项目列表等十项，任一失败即退出码非零。

· 语料：清单核对脚本按许可、指纹、字节数逐条核，当前 791 条记录、0 违规。

（三）当前的测量进度与已知问题

这里写清现在的真实状态，避免把“能演示”当成“已达标”。

表 3 三条硬指标的当前进度

指标	口径	当前状态
幻觉率	双人独立判读 + 第三人处理分歧后计算区间	打分包已生成并通过偏移核验（180/180 可切回原文）；两人独立标注尚未完成
画像—难度适配准确率	由标注者按画像与材料判断合适与否	判定表单已生成，同等人工标注进度
核心知识点覆盖率	知识点在原文中有带偏移的命中，按严格档统计	候选词表已挖出四轮清洗；知识点清单待领域人员签核

另有一个数据问题必须写在方案里：当前五个全文库中，22.0%（12,948 / 58,801）的证据片段实际上是论文末尾的参考文献条目。原因是建库脚本里“排除

参考文献段”的开关默认关闭，这批库是按默认参数建的。修法已经实现并带前后对照（排除后证据跨度下降 9.8%、跨文献重复簇下降 39%），但重建会覆盖五个库、需要重新解析 510 篇，目前没有擅自执行。这一条同时决定了界面上证据质量提示怎么写：照实标注，不做掩盖。

五、创新点

（一）把“接收一条关系”拆成四条可验证的判据

常见做法是用模型给出的置信度当接收阈值，问题是这个分数既不可解释也不可复现。我们把接收条件拆成证据有效、实体定位唯一、类型与方向合规、谓词在场四条，每条都能单独查、单独复算；置信度不再承担“是否接收”的职责，只作为排序参考。

（二）判定环节的确定性边界

裁判是代码不是模型：同样的输入必得同样的分数。它让“重跑一遍”这件事在系统里真的成立，也让争议可以停在具体条款上（哪一条判据没过），而不是停在“模型这么认为”。

（三）两种判断在结构上分开

用户的取舍写入独立的决定表，与机器状态互不覆盖；导出文件里两者各成列，评审者能直接看出哪些是机器判断、哪些是人做的判断。这一条在科研场景里比准确率更要紧。

（四）把负结果留在方案里

在 97 条真原文评测上，职责分离本身没有带来精度提升，增益来自检查项数量的增加。这个结论不好听，但它决定下一步该改抽取、而不是继续加角色。把它写进方案，是为了让后续工作有依据，而不是为了让阶段性成果看起来更好。

六、应用前景与商业化路径

（一）目标用户

· 课题组与研究生：做文献综述、开题调研，需要把“这个结论出自哪一篇哪一段”说清楚。
· 高校图书馆与科研管理部门：做学科情报与成果分析，需要可核验的关系数据而不是摘要。
· 企业技术情报岗位：做技术路线盘点，需要能把结论回溯到公开文献的原始表述。

（二）成本结构

系统默认走确定性规则与本地知识库，不依赖任何外部模型即可完成抽取与裁决，因此单次运行没有按次计费的成本；需要模型能力时按“用户自带 Key”处理，Key 只参与当次请求、不落盘、不进数据库。对客户而言，边际成本主要是语料采集与库重建，而不是调用费。

（三）推广路径

第一步先在团队内部与指导教师课题组做真实使用，把语料与标注补齐；第二步以“知识库 + 本地部署”的形式与一个院系或实验室合作，验证日常使用频次；第三步再考虑面向学科的版本化交付（按领域出库、按年更新）。当前的交付形态（单文件桌面版 + 局域网手机版）就是为“不联网也能用”准备的。

七、附录

（一）运行方式

本地起产品（开发用）：

```
powershell -ExecutionPolicy Bypass -File scripts/环境/启动产品.ps1 -NoBrowser
```

打包发行（生成单文件程序与两个发布包）：

```
打包软件.bat   # 产物在 release/ 下
```

跑一遍完整检查（编译 + 测试 + 覆盖率 + 前端类型与构建）：

```
powershell -ExecutionPolicy Bypass -File scripts/环境/运行全部测试.ps1
```

（二）目录结构与主要模块

表 4 仓库主要目录

路径	内容
src/yanhai/api.py	唯一产品后端，全部对外接口
src/yanhai/extraction.py	段落切分、实体与关系抽取、证据跨度
src/yanhai/agents.py	提出者、批判者、确定性裁判
src/yanhai/storage.py	账号、项目、关系决定、版本、成员权限
src/yanhai/knowledge_db.py	全文知识库只读视图
frontend/src/	八页工作区界面
scripts/抽取/构建全文知识库.py	五领域建库脚本（含参考文献段开关）
data/vertical_kb/	语料清单、领域注册表、知识卡
tests/	600 余项测试与公开实验协议

（三）数据与口径说明

语料只收许可证明确允许的开放全文（ACL Anthology 2016 年后、arXiv 条目页标注 CC、PubMed Central 开放获取子集、Europe PMC），每条都记录许可依据与文件指纹；论文正文只放 `papers/`，不入库，仓库里只保留题录、许可、

大小与 sha256。知识库统计、测试项数、打包体积等数字都可以用仓库内命令复算，复算方式写在各份说明文档里。